```
package Grafos;


public class Aresta{
    private Object elemento;
    private Vertice origem;
    private Vertice fim;
    private boolean direcionada;

    public Aresta() {
        this.elemento = null;
        this.origem = null;
        this.fim = null;
        this.direcionada = false;
    }

    public void setElement(Object elemento) {
        this.elemento = elemento;
    }

    public Object getElement() {
        return this.elemento;
    }

    public void setOrigem(Vertice origem) {
        this.origem = origem;
    }

    public Vertice getOrigem() {
        return this.origem;
    }

    public void setFim(Vertice fim) {
        this.fim = fim;
    }

    public Vertice getFim() {
        return this.fim;
    }

    public void setDirecionada(boolean direcionada) {
        this.direcionada = direcionada;
    }

    public boolean getDirecionada() {
        return this.direcionada;
    }


}
```

```
package Grafos;

public class Correcao extends RuntimeException{
    public Correcao(String erro){
        super(erro);
    }
}
```

```
package Grafos;
import java.util.ArrayList;

public class Grafo implements InterfaceGrafo{
    private Vertice[] vertices;
    private ArrayList<Aresta>[][] arestas;
    private int capacidade, size;

    public Grafo(int capacidade) {
        this.capacidade = capacidade;
        this.size = 0;
        this.vertices = new Vertice[capacidade];
        this.arestas = new ArrayList[capacidade][capacidade];

        //Por ser uma matriz de arestas, entÃ£o precisa definir o limite pela capacidad
e
        for (int i = 0; i < capacidade; i++){
            for (int j = 0; j < capacidade; j++){
                this.arestas[i][j] = new ArrayList<Aresta>();
            }
        }

    }

    public void removeVertice(Vertice v) throws Correcao {
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (v == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        for (int i = 0; i < vertices.length; i++){
            if (vertices[i] == v){

                //preciso tirar aresta que nao vai ter mais ligacao
                for (int j = 0; j < arestas.length; j++){
                    arestas[i][j].clear();
                    arestas[j][i].clear();
                }

                vertices[i] = null;
                size--;
                return;
            }

        }

        throw new Correcao("Vertice nÃ£o encontrado");
    }

    public void removeAresta(Aresta a) throws Correcao {
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        boolean encontrouAresta = false;

        //verifica a existencia da aresta no grafo, caso exista, remove a aresta
        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                if (arestas[i][j].remove(a)){
```

```
                    encontrouAresta = true;
                }
            }
        }

        if (!encontrouAresta){
            throw new Correcao("Aresta nÃ£o encontrada");
        }
    }

    public void insertVertice(Vertice v) throws Correcao {
        if (v == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        //vertice entra no grafo e vÃª um vazio para ser colocado na lista
        for (int i = 0; i < vertices.length; i++){
            if (vertices[i] == null){
                vertices[i] = v;
                size++;
                return;
            }
        }

        throw new Correcao("Grafo cheio");

    }

    public void insertAresta(Aresta a, Vertice v1, Vertice v2) throws Correcao {
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (v1 == null || v2 == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        if (a == null){
            throw new Correcao("Aresta nÃ£o encontrada");
        }

        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                if (vertices[i] == v1 && vertices[j] == v2){

                    //precisa definir origem e fim
                    a.setOrigem(v1);
                    a.setFim(v2);

                    if (a.getDirecionada() == true){
                        arestas[i][j].add(a);
                        return;
                    }

                    //sao do mesmo sentido
                    else{
                        arestas[i][j].add(a);

                        //se for laÃ§o (i == j), nÃ£o pode adicionar duas vezes na mesm
a cÃ©lula
                        if (i != j){
                            arestas[j][i].add(a);
                        }
```

```
                    return;
                }
            }
        }
    }

    throw new Correcao("Vertices nÃ£o encontrados");
}

public boolean isAdjacent(Vertice v1, Vertice v2, Aresta a) throws Correcao {
    if (isEmpty()){
        throw new Correcao("Grafo vazio");
    }

    if (v1 == null || v2 == null){
        throw new Correcao("Vertice nÃ£o encontrado");
    }

    if (a == null){
        throw new Correcao("Aresta nÃ£o encontrada");
    }

    for (int i = 0; i < arestas.length; i++){
        for (int j = 0; j < arestas[i].length; j++){
            if (vertices[i] == v1 && vertices[j] == v2){
                if (arestas[i][j].contains(a)){
                    return true;
                } else {
                    return false;
                }
            }
        }
    }

    throw new Correcao("Vertices nÃ£o encontrados");
}

public boolean isEmpty() {
    return size == 0;
}

public Vertice oposto(Vertice v, Aresta a) throws Correcao {
    if (isEmpty()){
        throw new Correcao("Grafo vazio");
    }

    if (v == null){
        throw new Correcao("Vertice nÃ£o encontrado");
    }

    if (a == null){
        throw new Correcao("Aresta nÃ£o encontrada");
    }

    if (a.getOrigem() == v){
        return a.getFim();

    }

    else if (a.getFim() == v){
        return a.getOrigem();
    }
```

```
        else {
            throw new Correcao("Vertice nÃ£o Ã© oposto da aresta");
        }

    }

    public Vertice[] finalVertices(Aresta a) throws Correcao {
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (a == null){
            throw new Correcao("Aresta nÃ£o encontrada");
        }

        //ele retorna um array de vertice que Ã© seu origem e fim que pertencem a mesma
 aresta
        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                if (arestas[i][j].contains(a)){
                    return new Vertice[]{a.getOrigem(), a.getFim()};
                }
            }
        }

        throw new Correcao("Aresta nÃ£o encontrada");

    }

    public void substituirVertice(Vertice v, Object elemento) throws Correcao {
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (v == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        for (int i = 0; i < vertices.length; i++){
            if (vertices[i] == v){
                vertices[i].setElement(elemento);
                return;
            }
        }

        throw new Correcao("Vertice nÃ£o encontrado");
    }

    public void substituirAresta(Aresta a, Object elemento) throws Correcao {
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (a == null){
            throw new Correcao("Aresta nÃ£o encontrada");
        }

        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                if (arestas[i][j].contains(a)){
                    a.setElement(elemento);
                    return;
                }
```

```
            }
        }

        throw new Correcao("Aresta nÃ£o encontrada");

    }

    public boolean isDirected(Aresta a) throws Correcao{
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (a == null){
            throw new Correcao("Aresta nÃ£o encontrada");
        }

        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                if (arestas[i][j].contains(a)){
                    return a.getDirecionada();
                }
            }
        }

        throw new Correcao("Aresta nÃ£o encontrada");
    }

    public void insertArestaDirecionada(Aresta a, Vertice v1, Vertice v2) throws Correc
ao{
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (a == null){
            throw new Correcao("Aresta nÃ£o encontrada");
        }

        if (v1 == null || v2 == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                if (vertices[i] == v1 && vertices[j] == v2){

                    //sÃ³ altera a aresta depois de confirmar que os vertices existem
                    a.setDirecionada(true);
                    a.setOrigem(v1);
                    a.setFim(v2);

                    arestas[i][j].add(a);
                    return;
                }
            }
        }


        throw new Correcao("Vertices nÃ£o encontrados");
    }

    public Vertice[] vertices() throws Correcao{
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
```

```
        }

        return vertices;
    }

    public ArrayList<Aresta>[][] arestas() throws Correcao{
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        return arestas;
    }

    public int grau(Vertice v) throws Correcao{
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (v == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        int grau = 0;
        ArrayList<Aresta> contadas = new ArrayList<Aresta>();

        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                for (Aresta a : arestas[i][j]){
                    if (v == a.getOrigem() || v == a.getFim()){
                        if (!contadas.contains(a)){

                            //laÃ§o conta 2 no grau
                            if (a.getOrigem() == a.getFim()){
                                grau += 2;
                            } else {
                                grau++;
                            }
                            contadas.add(a);
                        }

                    }
                }
            }
        }

        return grau;
    }

    public int grauEntrada(Vertice v) throws Correcao{
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (v == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        int grauEntrada = 0;

        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                for (Aresta a : arestas[i][j]){
```

```
                    if (v == a.getFim() && a.getDirecionada() == true){
                        grauEntrada++;

                    }
                }
            }
        }

        return grauEntrada;
    }

    public int grauSaida(Vertice v) throws Correcao{
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (v == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        int grauSaida = 0;

        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                for (Aresta a : arestas[i][j]){
                    if (a.getDirecionada() == true && v == a.getOrigem()){
                        grauSaida++;
                    }
                }
            }
        }

        return grauSaida;
    }

    public ArrayList<Aresta> arestasIncidentes(Vertice v) throws Correcao{
        if (isEmpty()){
            throw new Correcao("Grafo vazio");
        }

        if (v == null){
            throw new Correcao("Vertice nÃ£o encontrado");
        }

        ArrayList<Aresta> contadas = new ArrayList<Aresta>();

        for (int i = 0; i < arestas.length; i++){
            for (int j = 0; j < arestas[i].length; j++){
                for (Aresta a : arestas[i][j]){
                    if (v == a.getOrigem() || v == a.getFim()){
                        if (!contadas.contains(a)){
                            contadas.add(a);
                        }
                    }
                }
            }
        }

        return contadas;
    }

}
```

```
package Grafos;

import java.util.ArrayList;

public interface InterfaceGrafo{

    void removeVertice(Vertice v) throws Correcao;
    void removeAresta(Aresta a) throws Correcao;
    void insertVertice(Vertice v) throws Correcao;
    void insertAresta(Aresta a, Vertice v1, Vertice v2) throws Correcao;
    boolean isAdjacent(Vertice v1, Vertice v2, Aresta a) throws Correcao;
    boolean isEmpty();
    Vertice oposto(Vertice v, Aresta a) throws Correcao;
    Vertice[] finalVertices(Aresta a) throws Correcao;
    void substituirVertice(Vertice v, Object elemento) throws Correcao;
    void substituirAresta(Aresta a, Object elemento) throws Correcao;
    boolean isDirected(Aresta a) throws Correcao;
    void insertArestaDirecionada(Aresta a, Vertice v1, Vertice v2) throws Correcao;
    Vertice[] vertices() throws Correcao;
    ArrayList<Aresta>[][] arestas() throws Correcao;
    int grau(Vertice v) throws Correcao;
    int grauEntrada(Vertice v) throws Correcao;
    int grauSaida(Vertice v) throws Correcao;
    ArrayList<Aresta> arestasIncidentes(Vertice v) throws Correcao;
}
```

```
package Grafos;

import java.util.ArrayList;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Teste {

    public static void main(String[] args) {

        Scanner entrada = new Scanner(System.in);
        Grafo grafo = new Grafo(10);

        Vertice[] vertices = new Vertice[10];
        Aresta[] arestas = new Aresta[20];

        int opcao = -1;
        int quantidadeVertices = 0;
        int quantidadeArestas = 0;

        do {
            System.out.println("\n===== MENU DO GRAFO =====");
            System.out.println("1 - Inserir vertice");
            System.out.println("2 - Inserir aresta");
            System.out.println("3 - Remover vertice");
            System.out.println("4 - Remover aresta");
            System.out.println("5 - Verificar adjacencia");
            System.out.println("6 - Consultar vertice oposto");
            System.out.println("7 - Consultar vertices finais");
            System.out.println("8 - Substituir elemento de vertice");
            System.out.println("9 - Substituir elemento de aresta");
            System.out.println("10 - Verificar se o grafo esta vazio");
            System.out.println("11 - Verificar se aresta e direcionada");
            System.out.println("12 - Listar vertices");
            System.out.println("13 - Listar arestas");
            System.out.println("14 - Grau do vertice");
            System.out.println("15 - Grau de entrada do vertice");
            System.out.println("16 - Grau de saida do vertice");
            System.out.println("17 - Arestas incidentes ao vertice");
            System.out.println("0 - Sair");
            System.out.print("Escolha uma opcao: ");

            opcao = -1;

            try {
                opcao = entrada.nextInt();
                entrada.nextLine();

                switch (opcao) {

                    case 1: {
                        if (quantidadeVertices >= vertices.length) {
                            System.out.println("Limite de vertices do teste atingido.")
;
                            break;
                        }

                        System.out.print("Digite o elemento do vertice: ");
                        String elemento = entrada.nextLine();

                        Vertice v = new Vertice();
                        v.setElement(elemento);
```

```
                    grafo.insertVertice(v);
                    vertices[quantidadeVertices] = v;

                    System.out.println("Vertice inserido com sucesso. Indice: "
                            + quantidadeVertices);

                    quantidadeVertices++;
                    break;
                }

                case 2: {
                    if (quantidadeArestas >= arestas.length) {
                        System.out.println("Limite de arestas do teste atingido.");
                        break;
                    }

                    System.out.print("Indice do vertice de origem: ");
                    int origem = entrada.nextInt();

                    System.out.print("Indice do vertice de destino: ");
                    int destino = entrada.nextInt();
                    entrada.nextLine();

                    if (origem < 0 || origem >= vertices.length
                            || destino < 0 || destino >= vertices.length
                            || vertices[origem] == null
                            || vertices[destino] == null) {
                        System.out.println("Indice invalido ou vertice inexistente.
");
                        break;
                    }

                    System.out.print("Elemento da aresta: ");
                    String elemento = entrada.nextLine();

                    System.out.print("Aresta direcionada? (s/n): ");
                    String resposta = entrada.nextLine();

                    Aresta a = new Aresta();
                    a.setElement(elemento);

                    if (resposta.equalsIgnoreCase("s")) {
                        grafo.insertArestaDirecionada(a, vertices[origem], vertices
[destino]);
                    } else {
                        grafo.insertAresta(a, vertices[origem], vertices[destino]);
                    }

                    arestas[quantidadeArestas] = a;

                    System.out.println("Aresta inserida com sucesso. Indice: "
                            + quantidadeArestas);

                    quantidadeArestas++;
                    break;
                }

                case 3: {
                    System.out.print("Indice do vertice a remover: ");
                    int indice = entrada.nextInt();
                    entrada.nextLine();

                    if (indice < 0 || indice >= vertices.length
```

```
                    || vertices[indice] == null) {
                System.out.println("Vertice inexistente.");
                break;
            }

            Vertice removido = vertices[indice];
            grafo.removeVertice(removido);
            vertices[indice] = null;

            // o grafo apagou as arestas ligadas a esse vertice,
            // entao o teste tambem esquece essas arestas
            for (int k = 0; k < arestas.length; k++) {
                if (arestas[k] != null
                        && (arestas[k].getOrigem() == removido
                        || arestas[k].getFim() == removido)) {
                    arestas[k] = null;
                }
            }

            System.out.println("Vertice removido com sucesso.");
            break;
        }

        case 4: {
            System.out.print("Indice da aresta a remover: ");
            int indice = entrada.nextInt();
            entrada.nextLine();

            if (indice < 0 || indice >= arestas.length
                    || arestas[indice] == null) {
                System.out.println("Aresta inexistente.");
                break;
            }

            grafo.removeAresta(arestas[indice]);
            arestas[indice] = null;

            System.out.println("Aresta removida com sucesso.");
            break;
        }

        case 5: {
            System.out.print("Indice do primeiro vertice: ");
            int i = entrada.nextInt();

            System.out.print("Indice do segundo vertice: ");
            int j = entrada.nextInt();

            System.out.print("Indice da aresta: ");
            int k = entrada.nextInt();
            entrada.nextLine();

            if (i < 0 || i >= vertices.length
                    || j < 0 || j >= vertices.length
                    || k < 0 || k >= arestas.length
                    || vertices[i] == null
                    || vertices[j] == null
                    || arestas[k] == null) {
                System.out.println("Vertice ou aresta inexistente.");
                break;
            }

            System.out.println("Sao adjacentes: "
```

```
                        + grafo.isAdjacent(vertices[i], vertices[j], arestas[k]
));
                    break;
                }

                case 6: {
                    System.out.print("Indice do vertice: ");
                    int i = entrada.nextInt();

                    System.out.print("Indice da aresta: ");
                    int j = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= vertices.length
                            || j < 0 || j >= arestas.length
                            || vertices[i] == null || arestas[j] == null) {
                        System.out.println("Vertice ou aresta inexistente.");
                        break;
                    }

                    Vertice oposto = grafo.oposto(vertices[i], arestas[j]);
                    System.out.println("Vertice oposto: " + oposto.getElement());
                    break;
                }

                case 7: {
                    System.out.print("Indice da aresta: ");
                    int i = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= arestas.length || arestas[i] == null) {
                        System.out.println("Aresta inexistente.");
                        break;
                    }

                    Vertice[] finais = grafo.finalVertices(arestas[i]);

                    System.out.println("Vertice de origem: "
                            + finais[0].getElement());
                    System.out.println("Vertice de destino: "
                            + finais[1].getElement());
                    break;
                }

                case 8: {
                    System.out.print("Indice do vertice: ");
                    int i = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= vertices.length || vertices[i] == null) {
                        System.out.println("Vertice inexistente.");
                        break;
                    }

                    System.out.print("Novo elemento: ");
                    String elemento = entrada.nextLine();

                    grafo.substituirVertice(vertices[i], elemento);
                    System.out.println("Vertice atualizado.");
                    break;
                }

                case 9: {
```

```
                    System.out.print("Indice da aresta: ");
                    int i = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= arestas.length || arestas[i] == null) {
                        System.out.println("Aresta inexistente.");
                        break;
                    }

                    System.out.print("Novo elemento: ");
                    String elemento = entrada.nextLine();

                    grafo.substituirAresta(arestas[i], elemento);
                    System.out.println("Aresta atualizada.");
                    break;
                }

                case 10:
                    System.out.println("Grafo vazio: " + grafo.isEmpty());
                    break;

                case 11: {
                    System.out.print("Indice da aresta: ");
                    int i = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= arestas.length || arestas[i] == null) {
                        System.out.println("Aresta inexistente.");
                        break;
                    }

                    System.out.println("Direcionada: " + grafo.isDirected(arestas[i
]));
                    break;
                }

                case 12: {
                    Vertice[] lista = grafo.vertices();

                    for (int i = 0; i < lista.length; i++) {
                        if (lista[i] != null) {

                            // descobre o indice do vertice no teste
                            int indice = -1;
                            for (int k = 0; k < vertices.length; k++) {
                                if (vertices[k] == lista[i]) {
                                    indice = k;
                                }
                            }

                            System.out.println("Vertice " + indice + ": "
                                    + lista[i].getElement());
                        }
                    }
                    break;
                }

                case 13: {
                    ArrayList<Aresta>[][] matriz = grafo.arestas();
                    ArrayList<Aresta> vistas = new ArrayList<Aresta>();

                    for (int i = 0; i < matriz.length; i++) {
                        for (int j = 0; j < matriz[i].length; j++) {
```

```
                                for (Aresta a : matriz[i][j]) {
                                    if (!vistas.contains(a)) {
                                        vistas.add(a);

                                        // descobre o indice da aresta no teste
                                        int indice = -1;
                                        for (int k = 0; k < arestas.length; k++) {
                                            if (arestas[k] == a) {
                                                indice = k;
                                            }
                                        }

                                        System.out.println("Aresta " + indice
                                                + " [" + a.getElement() + "]: "
                                                + a.getOrigem().getElement()
                                                + (a.getDirecionada() ? " -> " : " -- "
)

                                                + a.getFim().getElement());
                                    }
                                }
                            }
                        }

                        if (vistas.isEmpty()) {
                            System.out.println("Nao ha arestas.");
                        }
                        break;
                    }

                    case 14: {
                        System.out.print("Indice do vertice: ");
                        int i = entrada.nextInt();
                        entrada.nextLine();

                        if (i < 0 || i >= vertices.length || vertices[i] == null) {
                            System.out.println("Vertice inexistente.");
                            break;
                        }

                        System.out.println("Grau: " + grafo.grau(vertices[i]));
                        break;
                    }

                    case 15: {
                        System.out.print("Indice do vertice: ");
                        int i = entrada.nextInt();
                        entrada.nextLine();

                        if (i < 0 || i >= vertices.length || vertices[i] == null) {
                            System.out.println("Vertice inexistente.");
                            break;
                        }

                        System.out.println("Grau de entrada: " + grafo.grauEntrada(vert
ices[i]));
                        break;
                    }

                    case 16: {
                        System.out.print("Indice do vertice: ");
                        int i = entrada.nextInt();
                        entrada.nextLine();
```

```
                        if (i < 0 || i >= vertices.length || vertices[i] == null) {
                            System.out.println("Vertice inexistente.");
                            break;
                        }

                        System.out.println("Grau de saida: " + grafo.grauSaida(vertices
[i]));
                        break;
                    }

                    case 17: {
                        System.out.print("Indice do vertice: ");
                        int i = entrada.nextInt();
                        entrada.nextLine();

                        if (i < 0 || i >= vertices.length || vertices[i] == null) {
                            System.out.println("Vertice inexistente.");
                            break;
                        }

                        ArrayList<Aresta> incidentes = grafo.arestasIncidentes(vertices
[i]);

                        if (incidentes.isEmpty()) {
                            System.out.println("Nenhuma aresta incidente.");
                        }

                        for (Aresta a : incidentes) {

                            // descobre o indice da aresta no teste
                            int indice = -1;
                            for (int k = 0; k < arestas.length; k++) {
                                if (arestas[k] == a) {
                                    indice = k;
                                }
                            }

                            System.out.println("Aresta " + indice + " [" + a.getElement
() + "]");
                        }
                        break;
                    }

                    case 0:
                        System.out.println("Encerrando o programa...");
                        break;

                    default:
                        System.out.println("Opcao invalida.");
                }

            } catch (Correcao e) {
                System.out.println("Erro: " + e.getMessage());

            } catch (InputMismatchException e) {
                // usuario digitou texto onde era numero
                entrada.nextLine();
                System.out.println("Entrada invalida. Digite um numero.");
            }

        } while (opcao != 0);

        entrada.close();
```

```
    }
}
```

```
package Grafos;


public class Vertice{
    private Object elemento;


    public Vertice() {
        this.elemento = null;

    }

    public void setElement(Object elemento) {
        this.elemento = elemento;
    }

    public Object getElement() {
        return this.elemento;
    }

}
```